

Unit 1 – Basics of Java & Introduction to OOP Paradigm

1. Introduction to Java

What is Java?

Java is a high-level, object-oriented, class-based programming language developed by **Sun Microsystems** in 1995. It is now maintained by Oracle and the Java community.

Java follows the principle:

"Write Once, Run Anywhere" (WORA)

This means that Java source code can be compiled into **bytecode**, which can run on any system having a compatible **Java Virtual Machine (JVM)**.

Simple Java Program

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

Output

Hello, Java!

2. Features of Java

Important features of Java are:

1. Simple

Java syntax is relatively easy to learn and removes complex features such as explicit pointer manipulation.

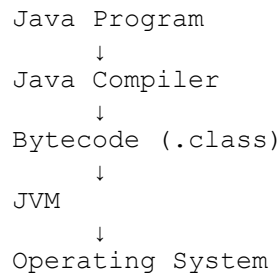
2. Object-Oriented

Java is based on OOP concepts such as:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

3. Platform Independent

Java source code is compiled into **bytecode**, which can execute on different operating systems using JVM.



4. Robust

Java provides:

- Strong type checking
- Exception handling
- Automatic garbage collection
- No direct pointer arithmetic

5. Secure

Java provides features such as:

- No direct pointer access
- Bytecode verification
- Runtime security checks

6. Portable

Java bytecode can be transferred between different platforms.

7. Multithreaded

Java supports executing multiple tasks simultaneously using threads.

8. Distributed

Java provides networking APIs for developing distributed applications.

9. High Performance

Java uses **JIT (Just-In-Time) compilation** to improve execution performance.

10. Dynamic

Java can dynamically load classes and libraries during runtime.

3. Java Bytecode

When a Java program is compiled, the Java compiler does **not directly generate machine code**.

It generates **bytecode**.

For example:

```
class Demo {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Compile:

```
javac Demo.java
```

This produces:

```
Demo.class
```

The `.class` file contains **bytecode**.

Run:

```
java Demo
```

Execution Process

```
Demo.java  
  |  
  | javac Demo.java  
  ↓  
Demo.class  
(Bytecode)  
  |  
  ↓  
JVM  
  |  
  ↓  
Operating System
```

Important

JDK → Compiler → Bytecode → JVM → Execution

4. Java Virtual Machine (JVM)

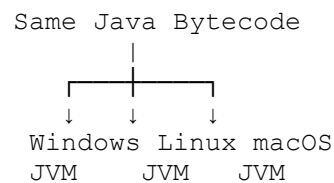
What is JVM?

JVM stands for Java Virtual Machine.

It is responsible for executing Java bytecode.

JVM is different for different operating systems, but the **bytecode remains the same**.

For example:

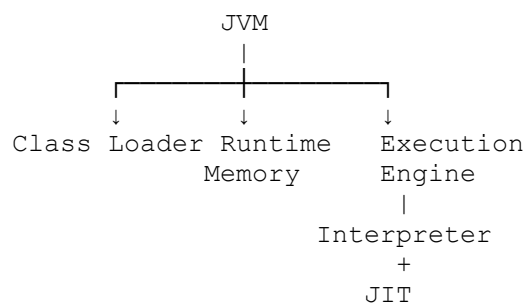


Main responsibilities of JVM

- Loads .class files
- Verifies bytecode
- Executes bytecode
- Manages memory
- Performs garbage collection

JVM Components

A simplified view:



5. JDK, JRE and JVM

These three terms are frequently confused.

JVM

Used to **execute Java bytecode**.

JRE

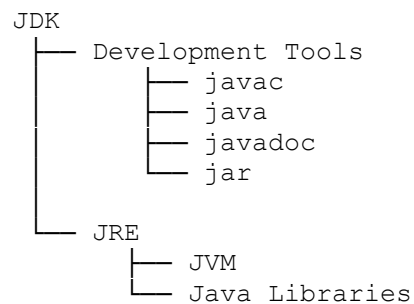
JRE = JVM + Java Runtime Libraries

Used to run Java applications.

JDK

JDK = JRE + Development Tools

Used to develop Java applications.



javac VS java

```
javac Demo.java
```

Compiles Java source code.

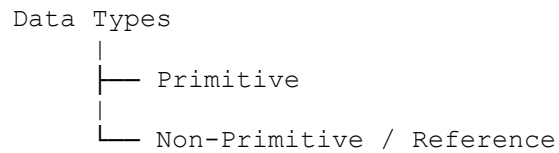
```
java Demo
```

Runs the compiled program.

6. Java Data Types

A **data type** specifies what type of data a variable can store.

Java data types are divided into:



Primitive Data Types

Java has 8 primitive data types.

Data Type	Size	Example
byte	8 bits	10
short	16 bits	1000
int	32 bits	50000
long	64 bits	500000L
float	32 bits	10.5f
double	64 bits	10.5
char	16 bits	'A'
boolean	JVM-dependent representation	true

7. Integer Data Types

Integer types store whole numbers.

byte

```
byte age = 20;
```

Range:

-128 to 127

short

```
short salary = 30000;
```

int

Most commonly used integer type.

```
int marks = 85;
```

long

Used for very large integer values.

```
long population = 1400000000L;
```

The `L` indicates a long literal.

8. Floating-Point Data Types

Used for decimal values.

float

```
float price = 25.50f;
```

`f` is required because decimal literals are `double` by default.

double

```
double pi = 3.14159;
```

Usually preferred when greater precision is required.

9. Character Data Type

`char` stores a single character.

```
char grade = 'A';
```

Character literals use **single quotes**.

```
char ch = 'X';
```

Not:

```
char ch = "X";    // Error
```

Unicode

Java `char` is a 16-bit type designed to represent a UTF-16 code unit.

```
char ch = 'A';
```

```
System.out.println(ch);
```

Output:

```
A
```

10. Boolean Data Type

Stores either:

```
true
```

or

```
false
```

Example:

```
boolean passed = true;
```

```
System.out.println(passed);
```

Output:

```
true
```

Java does **not** treat 0 and 1 as boolean values.

```
boolean x = 1;    // Error
```

11. Type Conversion

Type conversion means converting one data type into another.

There are two major forms:

1. Widening conversion
2. Narrowing conversion

11.1 Widening Conversion

Smaller compatible type → larger compatible type.

```
int x = 100;
double y = x;

System.out.println(y);
```

Output:

100.0

No explicit casting is generally required.

byte → short → int → long → float → double

12. Type Casting

Casting means explicitly converting a value from one type to another.

Narrowing Conversion

```
double x = 10.75;
int y = (int) x;

System.out.println(y);
```

Output:

10

The decimal portion is removed.

Another Example

```
int x = 130;
byte y = (byte) x;

System.out.println(y);
```

Output:

-126

Why?

Because `byte` has a limited range:

-128 to 127

The conversion wraps according to the binary representation.

13. Operators in Java

Operators are symbols used to perform operations on variables and values.

Major categories:

```
Operators
|
├── Arithmetic
├── Relational
├── Logical
├── Bitwise
├── Assignment
└── Other operators
```

14. Arithmetic Operators

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
int a = 10;
int b = 3;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Output:

```
13
7
30
3
1
```

Important: Integer Division

```
System.out.println(5 / 2);
```

Output:

2

Because both operands are integers.

But:

```
System.out.println(5.0 / 2);
```

Output:

2.5

15. Relational Operators

Relational operators compare two values.

Operator Meaning

==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

```
int a = 10;
int b = 20;

System.out.println(a < b);
System.out.println(a == b);
```

Output:

```
true
false
```

The result of a relational expression is a **boolean**.

16. Boolean Logical Operators

Operator Meaning

`&&` Logical AND
`,`
`!` Logical NOT

Example:

```
int age = 20;  
  
System.out.println(age >= 18 && age <= 60);
```

Output:

```
true
```

AND

```
true && true    → true  
true && false → false  
false && true  → false  
false && false → false
```

OR

```
true || true    → true  
true || false → true  
false || true  → true  
false || false → false
```

NOT

```
!true → false  
!false → true
```

17. Short-Circuit Evaluation

Java's `&&` and `||` operators use short-circuit evaluation.

Example:

```
int x = 10;  
  
if (x > 5 || x / 0 > 1) {  
    System.out.println("Hello");  
}
```

Output:

Hello

Why?

The first condition is already `true`, so Java does not evaluate the second condition.

Similarly:

```
if (x < 5 && x / 0 > 1)
```

The second condition is not evaluated because the first condition is `false`.

18. Bitwise Operators

Bitwise operators work at the level of individual bits.

Operator Meaning

<code>&</code>	Bitwise AND
<code>&</code>	<code>&</code>
<code>^</code>	Bitwise XOR
<code>~</code>	Bitwise complement
<code><<</code>	Left shift
<code>>></code>	Signed right shift
<code>>>></code>	Unsigned right shift

Example

```
int a = 5;
int b = 3;

System.out.println(a & b);
System.out.println(a | b);
System.out.println(a ^ b);
```

Binary:

```
5 = 101
3 = 011
```

AND:

```
101
011
---
001 = 1
```

OR:

```
101
011
---
111 = 7
```

XOR:

```
101
011
---
110 = 6
```

Output:

```
1
7
6
```

19. Assignment Operators

Basic assignment:

```
int x = 10;
```

Compound assignment operators:

```
+=
-=
*=
/=
%=
&=
|=
^=
<<=
>>=
>>>=
```

Example:

```
int x = 10;

x += 5;

System.out.println(x);
```

Output:

```
15
```

Equivalent to:

```
x = x + 5;
```

20. Operator Precedence

When multiple operators appear in an expression, Java follows precedence rules.

Example:

```
int result = 10 + 5 * 2;
```

Multiplication has higher precedence.

Therefore:

```
10 + (5 * 2)
= 20
```

Output:

```
20
```

Common order

A simplified order is:

```
()
unary operators
* / %
+ -
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= ...
```

Use parentheses for clarity

Instead of:

```
int x = a + b * c;
```

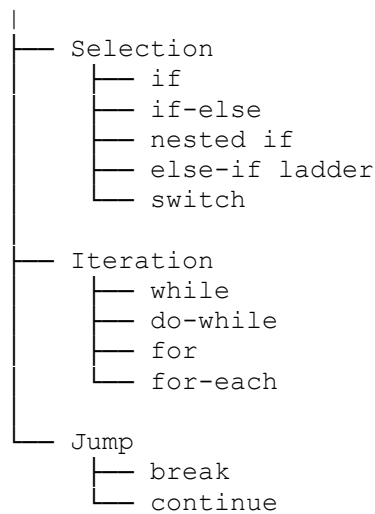
we can write:

```
int x = a + (b * c);
```

21. Control Statements

Control statements control the flow of program execution.

Control Statements



22. if Statement

Syntax:

```
if (condition) {  
    // statements  
}
```

Example:

```
int age = 20;  
  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
}
```

Output:

Eligible to vote

23. if-else

```
int marks = 45;  
  
if (marks >= 40) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

Output:

Pass

24. Nested if

An if statement inside another if statement.

```
int age = 25;
boolean citizen = true;

if (age >= 18) {
    if (citizen) {
        System.out.println("Eligible");
    }
}
```

Output:

Eligible

25. If-Else Ladder

Used when multiple conditions need to be checked.

```
int marks = 75;

if (marks >= 90) {
    System.out.println("A+");
} else if (marks >= 75) {
    System.out.println("A");
} else if (marks >= 60) {
    System.out.println("B");
} else if (marks >= 40) {
    System.out.println("C");
} else {
    System.out.println("Fail");
}
```

Output:

A

26. Switch Statement

Used when one expression needs to be compared against multiple cases.

```
int day = 2;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
}
```

```

        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Invalid day");
}

```

Output:

Tuesday

Why `break`?

Without `break`, execution may continue into the next cases.

27. Looping Statements

Loops are used to repeatedly execute a block of code.

Java provides:

- `while`
- `do-while`
- `for`
- enhanced `for` / `for-each`

28. While Loop

Syntax:

```

while (condition) {
    // statements
}

```

Example:

```

int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}

```

Output:

```

1
2
3
4
5

```

Important

The condition is checked **before** executing the loop body.

Therefore, a while loop can execute **zero times**.

29. Do-While Loop

Syntax:

```
do {  
    // statements  
} while (condition);
```

Example:

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

Output:

```
1  
2  
3  
4  
5
```

Important Difference

do-while executes the body **at least once**.

Example:

```
int i = 10;  
  
do {  
    System.out.println(i);  
} while (i < 5);
```

Output:

```
10
```

30. For Loop

Syntax:

```
for (initialization; condition; update) {  
    // statements  
}
```

Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

Execution:

```
initialization  
    ↓  
condition  
    ↓  
    body  
    ↓  
    update  
    ↓  
condition
```

31. For-Each Loop

Used mainly for arrays and collections.

Example:

```
int[] marks = {70, 80, 90, 85};  
  
for (int mark : marks) {  
    System.out.println(mark);  
}
```

Output:

```
70  
80  
90  
85
```

Syntax:

```
for (dataType variable : array) {  
    // statements  
}
```

Advantage

It is simpler when you only need the elements and don't need the index.

32. Jump Statements

Java provides:

- `break`
 - `continue`
-

33. `break`

Terminates the nearest loop or switch.

Example:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4
```

When `i` becomes 5, the loop terminates.

34. `continue`

Skips the current iteration and moves to the next iteration.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
  
    System.out.println(i);  
}
```

```
}
```

Output:

```
1
2
4
5
```

Difference

break	continue
Terminates loop	Skips current iteration
Control exits loop	Control goes to next iteration

35. Scope and Lifetime of Variables

Scope

Scope means the portion of a program where a variable can be accessed.

Lifetime

Lifetime means how long a variable exists in memory during program execution.

Local Variable

```
class Demo {
    void display() {
        int x = 10;

        System.out.println(x);
    }
}
```

`x` can only be accessed within `display()`.

Lifetime

The local variable exists while the method execution is active.

Block Scope

```
if (true) {
    int x = 10;
    System.out.println(x);
}
```

```
// System.out.println(x); // Error
```

x exists only inside the block.

Instance Variable

```
class Student {  
    int marks = 80;  
  
    void display() {  
        System.out.println(marks);  
    }  
}
```

marks belongs to an object.

Its lifetime is associated with the object.

Static Variable

```
class Student {  
    static int count = 0;  
}
```

A static variable belongs to the class rather than an individual object.

36. Introduction to OOP

OOP = Object-Oriented Programming

OOP organizes programs around **objects** rather than simply around functions.

Real-world examples:

```
Student  
Car  
Bank Account  
Employee  
Book  
Mobile Phone
```

A student can have:

```
Properties:  
name  
rollNo  
marks
```

```
Behaviors:
study()
attendClass()
giveExam()
```

This concept can be represented using a class.

37. Six Important Features of OOP

The six commonly taught features/principles are:

1. **Class**
2. **Object**
3. **Encapsulation**
4. **Inheritance**
5. **Polymorphism**
6. **Abstraction**

Some textbooks may list these slightly differently, but these six provide a useful introductory OOP framework.

38. Class

A **class** is a blueprint/template for creating objects.

Example:

```
class Student {

    String name;
    int marks;

    void display() {
        System.out.println(name);
        System.out.println(marks);
    }
}
```

Here:

```
Student
|
├── name
├── marks
└── display()
```

39. Fields

Variables declared inside a class are called **fields**.

```
class Student {  
  
    String name;  
    int age;  
}
```

Here:

```
name  
age
```

are fields.

Fields may be:

- Instance fields
- Static fields

40. Methods

A method defines behavior of a class.

```
class Student {  
  
    void study() {  
        System.out.println("Student is studying");  
    }  
}
```

Here:

```
study()
```

is a method.

41. Object

An **object** is an instance of a class.

Example:

```
Student s1 = new Student();
```

Here:

```
Student  
  ↑  
data type  
  
s1  
  ↑
```

```
reference variable  
  
new Student()  
  ↑  
object creation
```

42. `new` Operator

The `new` operator is used to create an object.

```
Student s1 = new Student();
```

It creates an object of `Student`.

Example:

```
class Student {  
    String name;  
    void display() {  
        System.out.println(name);  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "Rahul";  
        s1.display();  
    }  
}
```

Output:

```
Rahul
```

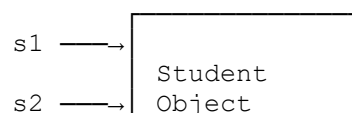
43. Assigning Object Reference Variables

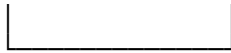
Consider:

```
Student s1 = new Student();  
Student s2 = s1;
```

There is only **one object**.

Both `s1` and `s2` refer to the same object.





Example:

```
class Student {
    int marks;
}

public class Test {

    public static void main(String[] args) {

        Student s1 = new Student();

        s1.marks = 50;

        Student s2 = s1;

        s2.marks = 90;

        System.out.println(s1.marks);
    }
}
```

Output:

90

Why?

Because `s1` and `s2` refer to the **same object**.

44. The Main Class and `main()` Method

The `main()` method is the standard entry point for a Java application launched by the Java launcher.

```
public static void main(String[] args)
```

Let's understand each keyword:

public

The method is accessible to the Java launcher.

static

The method can be invoked without creating an object of the class.

void

The method does not return a value.

main

The method name recognized as the application entry point.

String[] args

An array containing command-line arguments.

45. Command-Line Arguments

Command-line arguments are values passed to the program when it is executed.

Example:

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

Compile:

```
javac Test.java
```

Run:

```
java Test Hello Java
```

Output:

```
Hello  
Java
```

Here:

```
args[0] = "Hello"  
args[1] = "Java"
```

Important

Command-line arguments are stored as **Strings**.

Example:

```
java Test 10 20
```

The values are:

```
args[0] = "10"  
args[1] = "20"
```

They are not automatically integers.

To convert:

```
int a = Integer.parseInt(args[0]);
```

Example:

```
public class Add {  
  
    public static void main(String[] args) {  
  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
  
        System.out.println(a + b);  
    }  
}
```

Run:

```
java Add 10 20
```

Output:

```
30
```

46. `finalize()` Method

Historically, Java provided `Object.finalize()` as a method that the garbage collector could invoke before reclaiming an object.

Example often seen in older textbooks:

```
class Demo {  
  
    @Override  
    protected void finalize() {  
        System.out.println("Object destroyed");  
    }  
}
```

However, **important modern Java point:**

`finalize()` is deprecated and should not be used in new Java programs.

It is not a reliable mechanism for resource management because garbage collection timing is not guaranteed.

For resource cleanup, modern Java programs should generally use:

`try-with-resources`

and interfaces such as:

`AutoCloseable`

Exam point

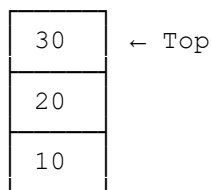
You may describe `finalize()` as an old mechanism associated with cleanup before garbage collection, but students should know that **it is deprecated in modern Java**.

47. A Stack Class

A **stack** is a linear data structure that follows:

LIFO – Last In, First Out

Example:



If 30 was inserted last, it will be removed first.

Common Stack Operations

Operation Meaning

<code>push()</code>	Add element
<code>pop()</code>	Remove top element
<code>peek()</code>	View top element
<code>empty()</code>	Check whether stack is empty
<code>search()</code>	Search an element

Java provides the legacy `Stack` class in:

```
java.util.Stack
```

Example of Stack

```
import java.util.Stack;

public class StackDemo {

    public static void main(String[] args) {

        Stack<Integer> stack = new Stack<>();

        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println(stack);
    }
}
```

```

        System.out.println("Top: " + stack.peek());

        System.out.println("Removed: " + stack.pop());

        System.out.println(stack);
    }
}

```

Output:

```

[10, 20, 30]
Top: 30
Removed: 30
[10, 20]

```

How it works

After:

```

stack.push(10);
stack.push(20);
stack.push(30);

```

Stack is:

```

30 ← Top
20
10

```

After:

```

stack.pop();

```

30 is removed.

```

20 ← Top
10

```

48. Complete Example Combining Several Concepts

```

class Student {

    String name;
    int marks;

    void displayResult() {

        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);

        if (marks >= 40) {
            System.out.println("Result: Pass");
        } else {
            System.out.println("Result: Fail");
        }
    }
}

```

```

    }
}

public class Test {

    public static void main(String[] args) {

        Student s1 = new Student();

        s1.name = "Amit";
        s1.marks = 75;

        s1.displayResult();
    }
}

```

Output:

```

Name: Amit
Marks: 75
Result: Pass

```

This single program demonstrates:

- Class
- Object
- Fields
- Method
- new
- Reference variable
- main()
- if-else
- Primitive data types

49. Quick Revision Table

Topic	Key Point
Java	High-level, object-oriented language
Bytecode	Intermediate code generated by Java compiler
JVM	Executes bytecode
JRE	JVM + runtime libraries
JDK	JRE + development tools
byte	8-bit signed integer
int	32-bit signed integer
long	64-bit signed integer
float	32-bit floating point
double	64-bit floating point
char	16-bit UTF-16 code unit
boolean	true or false
Widening	Smaller → larger compatible type
Casting	Explicit type conversion

Topic	Key Point
<code>if</code>	Conditional execution
<code>switch</code>	Multiple-choice selection
<code>while</code>	Condition checked before loop
<code>do-while</code>	Executes at least once
<code>for</code>	Initialization, condition, update
<code>for-each</code>	Iterates through elements
<code>break</code>	Terminates loop/switch
<code>continue</code>	Skips current iteration
Class	Blueprint
Object	Instance of class
Field	Variable in class
Method	Behavior/function in class
<code>new</code>	Creates object
Reference variable	Refers to an object
<code>main()</code>	Standard application entry point
<code>args</code>	Command-line arguments
Stack	LIFO data structure
<code>push()</code>	Adds to stack
<code>pop()</code>	Removes top
<code>peek()</code>	Returns top element

50. Important Questions for Students

Short-answer questions

1. What is Java?
2. Explain the meaning of "Write Once, Run Anywhere".
3. What is bytecode?
4. What is JVM?
5. Differentiate between JDK, JRE and JVM.
6. List the features of Java.
7. What are primitive data types?
8. Differentiate between `float` and `double`.
9. What is type casting?
10. What is widening conversion?
11. What are relational operators?
12. Differentiate between `&&` and `&`.
13. What is operator precedence?
14. What is an if-else ladder?
15. Differentiate between `while` and `do-while`.
16. Differentiate between `break` and `continue`.
17. What is scope of a variable?
18. What is a class?
19. What is an object?
20. What is the purpose of the `new` operator?
21. What are command-line arguments?

22. What is a reference variable?
23. What is a stack?
24. Explain LIFO.
25. What are `push()`, `pop()` and `peek()`?

Programming questions

1. Write a Java program to determine whether a number is positive, negative or zero.
2. Write a program to find the largest of three numbers.
3. Write a program to determine whether a number is even or odd.
4. Write a program to print numbers from 1 to 10 using a `for` loop.
5. Write a program to find the sum of numbers from 1 to 100.
6. Write a program to print the multiplication table of a number.
7. Write a program to demonstrate `break`.
8. Write a program to demonstrate `continue`.
9. Write a program using a `switch` statement to create a simple calculator.
10. Create a `Student` class containing fields `name` and `marks` and a method to display the student's details.
11. Demonstrate assigning one object reference to another.
12. Write a Java program using command-line arguments to add two numbers.
13. Write a program demonstrating widening and narrowing conversion.
14. Write a Java program demonstrating the `Stack` class.

Key conceptual question

Why is Java platform independent even though JVM is platform dependent?

Answer: Java source code is compiled into platform-independent bytecode. The bytecode is executed by a JVM, and each operating system has its own implementation of the JVM. Therefore, the same `.class` file can run on different platforms with compatible JVMs.